

Set 20: Creating, Loading, and Selecting Data with Pandas

Skill 20.1: Explain the purpose of pandas
Skill 20.2: Create a DataFrame from a dictionary
Skill 20.3: Create a DataFrame from a list
Skill 20.4: Load and save a CSV file
Skill 20.5: Inspect a DataFrame
Skill 20.6: Select columns from a DataFrame
Skill 20.7: Select rows from a DataFrame
Skill 20.8: Select rows with logic
Skill 20.9: Set row indices

Skill 20.1: Explain the purpose of Pandas

Skill 20.1 Concepts

Pandas is a Python module for working with tabular data (i.e., data in a table with rows and columns). Tabular data has a lot of the same functionality as Excel, but pandas adds the power of Python.

To get access to the pandas module, it requires that the module be installed. If you have Anaconda installed, pandas is already installed.

The pandas is usually imported at the top of a Python file under the alias `pd`,

```
import pandas as pd
```

If we need to access the pandas module, we can do so by operating on `pd`.

In this lesson, you'll learn the basics of working with a single table in pandas. More specifically, you will learn how to,

- Create a table from scratch
- Load data from another file
- Select certain rows or columns of a table

Skill 20.2: Create a DataFrame from a dictionary

Skill 20.2 Concepts

A DataFrame is an object that stores data as rows and columns. You can think of a DataFrame as a spreadsheet or as a Google Sheets table. You can manually create a DataFrame or fill it with data from a CSV, or an Excel spreadsheet.

DataFrames have rows and columns. Each column has a name, which is a string. Each row has an index, which is an integer. DataFrames can contain many different data types: strings, ints, floats, tuples, etc.

You can pass in a dictionary to `pd.DataFrame()`. Each key is a column name and each value is a list of column values. The columns must all be the same length or you will get an error. Below is an example,

Code

```
import pandas as pd

my_dictionary = {
    'name': ['John Smith', 'Jane Doe', 'Joe Schmo'],
    'address': ['123 Main St.', '456 Maple Ave.', '789 Broadway'],
    'age': [34, 28, 51]
}

my_df = pd.DataFrame(my_dictionary)
print(my_df)
```

Output

	name	address	age
0	John Smith	123 Main St.	34
1	Jane Doe	456 Maple Ave.	28
2	Joe Schmo	789 Broadway	51

Skill 20.2 Exercise 1

Skill 20.3: Create a DataFrame from a list

Skill 20.3 Concepts

You can also add data to a DataFrame using lists.

For example, you can pass in a list of lists, where each one represents a row of data. The columns keyword argument can be used to pass a list of column names.

Code

```
my_list = [  
    ['John Smith', '123 Main St.', 34],  
    ['Jane Doe', '456 Maple Ave.', 28],  
    ['Joe Schmo', '789 Broadway', 51]  
]  
fields=['name', 'address', 'age']  
my_df = pd.DataFrame(my_list, columns = fields)  
print(my_df)
```

Output

	name	address	age
0	John Smith	123 Main St.	34
1	Jane Doe	456 Maple Ave.	28
2	Joe Schmo	789 Broadway	51

Skill 20.3 Exercise 1

Skill 20.4: Load and save a CSV file

Skill 20.4 Concepts

CSV (*comma separated values*) is a text-only spreadsheet format. You can find CSVs in lots of places:

- Online datasets (here's an example from data.gov)
- Export from Excel or Google Sheets
- Export from SQL

The first row of a CSV contains column headings. All subsequent rows contain values. Each column heading and each variable is separated by a comma:

```
column1,column2,column3  
value1,value2,value3
```

Below is an example with real data,

```
name, cake_flavor, frosting_flavor, topping  
Chocolate Cake, chocolate, chocolate, chocolate shavings  
Birthday Cake, vanilla, vanilla, rainbow sprinkles  
Carrot Cake, carrot, cream, cheese almonds
```

When you have data in a CSV, you can load it into a DataFrame in Pandas using the `read_csv()` method.

csv_data.csv				
	name	cake_flavor	frosting_flavor	topping
	Chocolate Cake	chocolate	chocolate	chocolate shavings
	Birthday Cake	vanilla	vanilla	rainbow sprinkles
	Carrot Cake	carrot	cream	cheese almonds
Code				
<pre>df = pd.read_csv('csv_data.csv') print(df)</pre>				
Output				
	name	cake_flavor	frosting_flavor	topping
0	Chocolate Cake	chocolate	chocolate	chocolate shavings
1	Birthday Cake	vanilla	vanilla	rainbow sprinkles
2	Carrot Cake	carrot	cream	cheese almonds

We can also save data to a CSV, using the `to_csv()` method

DataFrame df				
	name	cake_flavor	frosting_flavor	topping
0	Chocolate Cake	chocolate	chocolate	chocolate shavings
1	Birthday Cake	vanilla	vanilla	rainbow sprinkles
2	Carrot Cake	carrot	cream	cheese almonds
Code				
<pre>df.to_csv('new_csv_data.csv')</pre>				
new_csv_data.csv				
	name	cake_flavor	frosting_flavor	topping
	Chocolate Cake	chocolate	chocolate	chocolate shavings
	Birthday Cake	vanilla	vanilla	rainbow sprinkles
	Carrot Cake	carrot	cream	cheese almonds

In the example above, the `to_csv()` method is called on `df` (which represents a DataFrame object). The name of the CSV file is passed in as an argument (`new_csv_data.csv`). By default, this method will save the CSV file in your current directory.

[Skill 20.4 Exercise 1](#)

Skill 20.5: Inspect a DataFrame

Skill 20.5 Concepts

When we load a new DataFrame from a CSV, we want to know what it looks like.

If it's a small DataFrame, you can display it by typing `print(df)`.

If it's a larger DataFrame, it's helpful to be able to inspect a few items without having to look at the entire DataFrame.

The method `head()` gives the first 5 rows of a DataFrame. If you want to see more rows, you can pass in the positional argument `n`. For example, `df.head(10)` would show the first 10 rows.

The method `info()` gives some statistics for each column.

csv_data.csv				
name,cake_flavor,frosting_flavor,topping Chocolate Cake,chocolate,chocolate,chocolate shavings Birthday Cake,vanilla,vanilla,rainbow sprinkles Carrot Cake,carrot,cream,cheese almonds Oreo Cake,chocolate,vanilla,oreo cookies Lemon Cake,lemon,vanilla,candied lemon rind German Chocolate Cake,chocolate salted caramel,coconut-pecan,pecans				
Code				
df = pd.read_csv('csv_data.csv')				
print(df.head())				
Output				
	name	cake_flavor	frosting_flavor	topping
0	Chocolate Cake	chocolate	chocolate	chocolate shavings
1	Birthday Cake	vanilla	vanilla	rainbow sprinkles
2	Carrot Cake	carrot	cream	cheese almonds
3	Oreo Cake	chocolate	vanilla	oreo cookies
4	Lemon Cake	lemon	vanilla	candied lemon rind
Code				
print(df.head(2))				
Output				
	name	cake_flavor	frosting_flavor	topping
0	Chocolate Cake	chocolate	chocolate	chocolate shavings
1	Birthday Cake	vanilla	vanilla	rainbow sprinkles
Code				
print(df.info())				
Output				
<class 'pandas.core.frame.DataFrame'> RangeIndex: 6 entries, 0 to 5 Data columns (total 4 columns): # Column Non-Null Count Dtype --- ----- 0 name 6 non-null object 1 cake_flavor 6 non-null object 2 frosting_flavor 6 non-null object 3 topping 6 non-null object dtypes: object(4) memory usage: 324.0+ bytes None				

Skill 20.5 Exercise 1

Skill 20.6: Select columns from a DataFrame

Skill 20.6 Concepts

Now we know how to create and load data. Let's select parts of those datasets that are interesting or important to our analyses.

Suppose you have the DataFrame called *cakes*,

	name	cake_flavor	frosting_flavor	topping
0	Chocolate Cake	chocolate	chocolate	chocolate shavings
1	Birthday Cake	vanilla	vanilla	rainbow sprinkles
2	Carrot Cake	carrot	cream	cheese almonds
3	Oreo Cake	chocolate	vanilla	oreo cookies
4	Lemon Cake	lemon	vanilla	candied lemon rind

Perhaps you want to take the average or plot a histogram of a column. To do either of these tasks, you'd need to select the column.

There are two possible syntaxes for selecting all values from a column,

1. Select the column as if you were selecting a value from a dictionary using a key. In our example, we would type `cakes['name']` to select the ages.
2. If the name of a column follows all the rules for a variable name (doesn't start with a number, doesn't contain spaces or special characters, etc.), then you can select it using the following notation: `df.name_of_column_to_select`. In our example, we would type `cakes.name`.

DataFrame cakes				
	name	cake_flavor	frosting_flavor	topping
0	Chocolate Cake	chocolate	chocolate	chocolate shavings
1	Birthday Cake	vanilla	vanilla	rainbow sprinkles
2	Carrot Cake	carrot	cream	cheese almonds
3	Oreo Cake	chocolate	vanilla	oreo cookies
4	Lemon Cake	lemon	vanilla	candied lemon rind
Code				
<pre>print(cakes["topping"])</pre>				
Output				
0	chocolate shavings			
1	rainbow sprinkles			
2	cheese almonds			
3	oreo cookies			
4	candied lemon rind			
Code				
<pre>print(cakes.topping)</pre>				
Output				
0	chocolate shavings			
1	rainbow sprinkles			
2	cheese almonds			
3	oreo cookies			
4	candied lemon rind			

When you have a larger DataFrame, you might want to select just a few columns.

For instance, check out the DataFrame of orders from ShoeFly.com,

id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red

We might just be interested in the customer's *last_name* and *email*. We want a DataFrame like this,

last_name	email
Lindsay	RebeccaLindsay57@hotmail.com
Joyce	EmilyJoyce25@gmail.com
Waller	Joyce.Waller@gmail.com
Erickson	Justin.Erickson@outlook.com

To select two or more columns from a DataFrame, we use a list of the column names.

DataFrame orders						
id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red
Code						
<pre>new_df = orders[['last_name', 'email']]</pre>						
DataFrame new_df						
last_name	email					
Lindsay	RebeccaLindsay57@hotmail.com					
Joyce	EmilyJoyce25@gmail.com					
Waller	Joyce.Waller@gmail.com					
Erickson	Justin.Erickson@outlook.com					

Note: Make sure that you have a double set of brackets ([][]), or this command won't work!

Skill 20.6 Exercises 1 & 2

Skill 20.7: Select rows from a DataFrame

Skill 20.7 Concepts

Let's revisit our orders from ShoeFly.com,

id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red

Maybe our Customer Service department has just received a message from Joyce Waller, so we want to know exactly what she ordered. We want to select this single row of data.

DataFrames are zero-indexed, meaning that we start with the 0th row and count up from there. Joyce Waller's order is the 2nd row. The example below illustrates how we can select it,

DataFrame orders						
id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red
Code						
<pre>row_2 = df.iloc[2] print(row_2)</pre>						
DataFrame new_df						
id	91987					
first_name	Joyce					
last_name	Waller					
email	Joyce.Waller@gmail.com					
shoe_type	sandals					
shoe_material	fabric					
shoe_color	black					
Name: 2, dtype: object						

You can also select multiple rows from a DataFrame.

Below some examples from the ShoeFly.com's orders DataFrame.

DataFrame orders						
id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red
79357	Andrew	Banks	AB4318@gmail.com	boots	leather	brown
52386	Julie	Marsh	JulieMarsh59@gmail.com	sandals	fabric	black
20487	Thomas	Jensen	TJ5470@gmail.com	clogs	fabric	navy
76971	Janice	Hicks	Janice.Hicks@gmail.com	clogs	faux-leather	navy
21586	Gabriel	Porter	GabrielPorter24@gmail.com	clogs	leather	brown

Code	Explanation
<code>orders.iloc[3:7]</code>	Selects all rows starting at the 3rd row and up to but <i>not including</i> the 7th row (i.e., the 3rd row, 4th row, 5th row, and 6th row)

Output						
id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red
79357	Andrew	Banks	AB4318@gmail.com	boots	leather	brown
52386	Julie	Marsh	JulieMarsh59@gmail.com	sandals	fabric	black
20487	Thomas	Jensen	TJ5470@gmail.com	clogs	fabric	navy

Code	Explanation
<code>orders.iloc[:4]</code>	Selects all rows up to, but <i>not including</i> the 4th row (i.e., the 0th, 1st, 2nd, and 3rd rows)

Output						
id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
54791	Rebecca	Lindsay	RebeccaLindsay57@hotmail.com	clogs	faux-leather	black
53450	Emily	Joyce	EmilyJoyce25@gmail.com	ballet flats	faux-leather	navy
91987	Joyce	Waller	Joyce.Waller@gmail.com	sandals	fabric	black
14437	Justin	Erickson	Justin.Erickson@outlook.com	clogs	faux-leather	red

Code	Explanation
<code>orders.iloc[-3:]</code>	Selects all rows up to, but <i>not including</i> the 4th row (i.e., the 0th, 1st, 2nd, and 3rd rows)

Output						
id	first_name	last_name	email	shoe_type	shoe_material	shoe_color
20487	Thomas	Jensen	TJ5470@gmail.com	clogs	fabric	navy
76971	Janice	Hicks	Janice.Hicks@gmail.com	clogs	faux-leather	navy
21586	Gabriel	Porter	GabrielPorter24@gmail.com	clogs	leather	brown

Skill 20.7 Exercises 1

Skill 20.8: Select rows with logic

Skill 20.8 Concepts

You can select a subset of a DataFrame by using the Boolean logical statements we've learned about previously. For example,

```
df[df.MyColumnName == desired_column_value]
```

Below are some examples,

DataFrame orders																																
<table><thead><tr><th>name</th><th>address</th><th>phone</th><th>age</th></tr></thead><tbody><tr><td>Martha Jones</td><td>123 Main St.</td><td>234-567-8910</td><td>28</td></tr><tr><td>Rose Tyler</td><td>456 Maple Ave.</td><td>212-867-5309</td><td>22</td></tr><tr><td>Donna Noble</td><td>789 Broadway</td><td>949-123-4567</td><td>35</td></tr><tr><td>Amy Pond</td><td>98 West End Ave.</td><td>646-555-1234</td><td>29</td></tr><tr><td>Clara Oswald</td><td>54 Columbus Ave.</td><td>714-225-1957</td><td>31</td></tr><tr><td>...</td><td>...</td><td>...</td><td>...</td></tr></tbody></table>				name	address	phone	age	Martha Jones	123 Main St.	234-567-8910	28	Rose Tyler	456 Maple Ave.	212-867-5309	22	Donna Noble	789 Broadway	949-123-4567	35	Amy Pond	98 West End Ave.	646-555-1234	29	Clara Oswald	54 Columbus Ave.	714-225-1957	31	...	...	...	...	
name	address	phone	age																													
Martha Jones	123 Main St.	234-567-8910	28																													
Rose Tyler	456 Maple Ave.	212-867-5309	22																													
Donna Noble	789 Broadway	949-123-4567	35																													
Amy Pond	98 West End Ave.	646-555-1234	29																													
Clara Oswald	54 Columbus Ave.	714-225-1957	31																													
...	...	...	...																													
Code		Explanation																														
<pre>over_30 = df[df.age == 30] print(over_30)</pre>		An empty DataFrame is returned because there are no customers with equal to 30																														
Output																																
Empty DataFrame Columns: [name, address, phone, age] Index: []																																
Code		Explanation																														
<pre>over_30 = df[df.age > 30] print(over_30)</pre>		Returns a DataFrame with the customers over 30																														
Output																																
	name	address	phone	age																												
2	Donna Noble	789 Broadway	949-123-4567	35																												
3	Amy Pond	98 West End Ave.	208-580-8760	31																												
4	Clara Oswald	54 Columbus Ave.	714-225-1957	33																												
Code		Explanation																														
<pre>c = df[df.name != 'Clara Oswald'] print(c)</pre>		Returns a DataFrame of all the customers not named <i>Clara Oswald</i>																														
Output																																
	name	address	phone	age																												
0	Martha Jones	123 Main St	234-567-8910	28																												
1	Rose Tyler	456 Maple Ave.	212-867-5309	22																												
2	Donna Noble	789 Broadway	949-123-4567	35																												
3	Amy Pond	98 West End Ave.	208-580-8760	31																												

You can also combine multiple logical statements, as long as each statement is in parentheses.

For instance, suppose we wanted to select all rows where the customer's age was under 30 *or* the customer's name was "Martha Jones",

DataFrame orders				
name	address	phone	age	
Martha Jones	123 Main St.	234-567-8910	28	
Rose Tyler	456 Maple Ave.	212-867-5309	22	
Donna Noble	789 Broadway	949-123-4567	35	
Amy Pond	98 West End Ave.	646-555-1234	29	
Clara Oswald	54 Columbus Ave.	714-225-1957	31	
...	...	...	...	

Code	Explanation
<pre>results = df[(df.age < 30) (df.name == 'Martha Jones')] print(results)</pre>	Returns a DataFrame of the customer's whose age is under 30 <i>or</i> whose name is "Martha Jones":

Output				
	name	address	phone	age
0	Martha Jones	123 Main St	234-567-8910	28
1	Rose Tyler	456 Maple Ave.	212-867-5309	22

Code	Explanation
<pre>results = df[df.name.isin(['Martha Jones', 'Rose Tyler', 'Amy Pond'])]</pre> <pre>print(results)</pre>	Uses the isin command to return a DataFrame of the customers whose name appears in a list of values.

Output				
	name	address	phone	age
0	Martha Jones	123 Main St	234-567-8910	28
1	Rose Tyler	456 Maple Ave.	212-867-5309	22
3	Amy Pond	98 West End Ave.	208-580-8760	31

Skill 20.8 Exercise 1

Skill 20.9: Set row indices

Skill 20.9 Concepts

When we select a subset of a DataFrame using logic, we end up with non-consecutive indices. This is inelegant and makes it hard to use .iloc().

Example below illustrates this problem.

<pre>results = df[df.name.isin(['Martha Jones', 'Rose Tyler', 'Amy Pond'])] print(results)</pre>	Uses the isin command to return a DataFrame of the customers whose name appears in a list of values. Notice the output contains non-consecutive indices																				
Output																					
<table><tr><th></th><th>name</th><th>address</th><th>phone</th><th>age</th></tr><tr><td>0</td><td>Martha Jones</td><td>123 Main St</td><td>234-567-8910</td><td>28</td></tr><tr><td>1</td><td>Rose Tyler</td><td>456 Maple Ave.</td><td>212-867-5309</td><td>22</td></tr><tr><td>3</td><td>Amy Pond</td><td>98 West End Ave.</td><td>208-580-8760</td><td>31</td></tr></table>			name	address	phone	age	0	Martha Jones	123 Main St	234-567-8910	28	1	Rose Tyler	456 Maple Ave.	212-867-5309	22	3	Amy Pond	98 West End Ave.	208-580-8760	31
	name	address	phone	age																	
0	Martha Jones	123 Main St	234-567-8910	28																	
1	Rose Tyler	456 Maple Ave.	212-867-5309	22																	
3	Amy Pond	98 West End Ave.	208-580-8760	31																	

We can fix this using the method `reset_index()`. For example, here is a DataFrame called `df` with non-consecutive indices,

<pre>results = df[df.name.isin(['Martha Jones', 'Rose Tyler', 'Amy Pond'])] results2 = results.reset_index() print(results2)</pre>	If we use the command <code>df.reset_index()</code>, we get a new DataFrame with a new set of indices. Notice, the old indices are stored in a new column called <i>index</i>.																								
Output																									
<table><tr><th></th><th>index</th><th>name</th><th>address</th><th>phone</th><th>age</th></tr><tr><td>0</td><td>0</td><td>Martha Jones</td><td>123 Main St</td><td>234-567-8910</td><td>28</td></tr><tr><td>1</td><td>1</td><td>Rose Tyler</td><td>456 Maple Ave.</td><td>212-867-5309</td><td>22</td></tr><tr><td>2</td><td>3</td><td>Amy Pond</td><td>98 West End Ave.</td><td>208-580-8760</td><td>31</td></tr></table>			index	name	address	phone	age	0	0	Martha Jones	123 Main St	234-567-8910	28	1	1	Rose Tyler	456 Maple Ave.	212-867-5309	22	2	3	Amy Pond	98 West End Ave.	208-580-8760	31
	index	name	address	phone	age																				
0	0	Martha Jones	123 Main St	234-567-8910	28																				
1	1	Rose Tyler	456 Maple Ave.	212-867-5309	22																				
2	3	Amy Pond	98 West End Ave.	208-580-8760	31																				

Notice in the above example, the old indices have been moved into a new column called 'index'. Unless you need those values for something special, it's probably better to use the keyword `drop=True` so that you don't end up with that extra column.

<pre>results = df[df.name.isin(['Martha Jones', 'Rose Tyler', 'Amy Pond'])] results2 = results.reset_index(drop = True) print(results2)</pre>	Adding the keyword <code>drop = True</code> removes the extra column.																				
Output																					
<table><tr><th></th><th>name</th><th>address</th><th>phone</th><th>age</th></tr><tr><td>0</td><td>Martha Jones</td><td>123 Main St</td><td>234-567-8910</td><td>28</td></tr><tr><td>1</td><td>Rose Tyler</td><td>456 Maple Ave.</td><td>212-867-5309</td><td>22</td></tr><tr><td>2</td><td>Amy Pond</td><td>98 West End Ave.</td><td>208-580-8760</td><td>31</td></tr></table>			name	address	phone	age	0	Martha Jones	123 Main St	234-567-8910	28	1	Rose Tyler	456 Maple Ave.	212-867-5309	22	2	Amy Pond	98 West End Ave.	208-580-8760	31
	name	address	phone	age																	
0	Martha Jones	123 Main St	234-567-8910	28																	
1	Rose Tyler	456 Maple Ave.	212-867-5309	22																	
2	Amy Pond	98 West End Ave.	208-580-8760	31																	

Using `reset_index()` will return a new DataFrame, but we usually just want to modify our existing DataFrame. If we use the keyword `inplace=True` we can just modify our existing DataFrame.

```

results = df[df.name.isin(['Martha Jones',
                            'Rose Tyler',
                            'Amy Pond'])]

results.reset_index(drop = True, inplace = True)
print(results)

```

Adding the keyword `inplace = True` allows us to modify the current DataFrame without creating a new one.

Output

	name	address	phone	age
0	Martha Jones	123 Main St	234-567-8910	28
1	Rose Tyler	456 Maple Ave.	212-867-5309	22
2	Amy Pond	98 West End Ave.	208-580-8760	31

Skill 20.9 Exercise 1